Health Probes & Application Debugging

Keeping Applications Resilient & Diagnosable — Module 10 of 13

Enterprise EKS Platform

📋 Module Agenda

- Why Health Checks Matter
- Probe Types: Liveness, Readiness, Startup
- Probe Configuration & Parameters
- HTTP, TCP, Exec, and gRPC Probes
- Probe Best Practices & Anti-Patterns
- Graceful Shutdown & Pod Disruption Budgets
- Debugging Probe Failures

💓 Why Health Checks Matter

From Process Status to Application Health

🔄 Pod Lifecycle & Restart Policies

Pod Phase Progression

Pending → Running → Succeeded / Failed

- **Always:** Restart container regardless of exit code (default for Deployments)
- **OnFailure:** Restart only on non-zero exit (common for Jobs)
- **Never:** Never restart — let the pod enter Failed state

> **Key Insight:** Restart policies only respond to *process exits* — they cannot detect a hung process, a deadlock, or a broken database connection.

💥 What Happens Without Probes

🔴 Cascading Failure Scenario

1. App loses database connection
2. Process stays running, returns 500s
3. Service keeps routing traffic to unhealthy pod
4. Users experience errors across the cluster
5. Load balancer spreads failure further

✅ With Probes Configured

1. App loses database connection
2. Readiness probe fails → pod removed from Service
3. Liveness probe fails → container restarted
4. Traffic routed only to healthy pods
5. Self-healing with zero manual intervention

🔍 Probe Types

Liveness, Readiness, and Startup

💚 Liveness Probes

Purpose: Detect when a container is *stuck* and needs to be *restarted*.
Answers: "Is the application still alive?"

```yaml
containers:
- name: app
  image: myapp:1.0
  livenessProbe:
    httpGet:
      path: /healthz
      port: 8080
    initialDelaySeconds: 15
    periodSeconds: 10
    failureThreshold: 3
```

- Failure action: **kubelet restarts the container**
- Use for deadlocks, infinite loops, unrecoverable states
- Should check *internal* health only — not external dependencies

💚 Liveness: TCP & Exec Variants

TCP Socket Probe

```
livenessProbe:
  tcpSocket:
    port: 3306
  initialDelaySeconds: 30
  periodSeconds: 10
```

Succeeds if TCP connection opens. Ideal for databases, message brokers.

Exec Command Probe

```
livenessProbe:
  exec:
    command:
    - /bin/sh
    - -c
    - pg_isready -U postgres
  initialDelaySeconds: 30
  periodSeconds: 15
```

Runs command inside container. Exit code 0 = success.

🔵 Readiness Probes

Purpose: Control when a pod *receives traffic* from a Service. Answers: "Is the application ready to accept requests?"

```yaml
containers:
- name: app
  image: myapp:1.0
  readinessProbe:
    httpGet:
      path: /ready
      port: 8080
    periodSeconds: 5
    failureThreshold: 3
```

- Failure action: **pod removed from Service endpoints** (not restarted)
- Can check downstream dependencies (caches warmed, connections established)
- Pod stays running — can recover and rejoin the Service

🔷 Startup Probes

Purpose: Protect slow-starting containers from being killed by liveness probes before they're ready.

```yaml
containers:
- name: app
  image: legacy-app:2.0
  startupProbe:
    httpGet: { path: /healthz, port: 8080 }
    failureThreshold: 30
    periodSeconds: 10
  livenessProbe:
    httpGet: { path: /healthz, port: 8080 }
    periodSeconds: 10
```

- Liveness and readiness probes **disabled until startup probe succeeds**
- Max startup time = `failureThreshold × periodSeconds` (300s above)
- Ideal for legacy apps, Java apps with long JVM warmup, ML model loading

📈 Probe Comparison

Aspect	Liveness	Readiness	Startup
Question	Is it alive?	Can it serve traffic?	Has it finished starting?
On Failure	Restart container	Remove from Service	Restart container
Check Dependencies?	No	Yes	No
When Active	After startup succeeds	After startup succeeds	Container start only
Default if Omitted	Assumed alive	Assumed ready	Assumed started

> **Recommended:** Use all three probes together. Startup handles initialization, liveness catches deadlocks, readiness controls traffic flow.

Probe Configuration

Parameters, Protocols, and Tuning

⚙️ Probe Parameters

Parameter	Default	Description
`initialDelaySeconds`	0	Seconds to wait before first probe
`periodSeconds`	10	Interval between consecutive probes
`timeoutSeconds`	1	Max seconds to wait for probe response
`successThreshold`	1	Consecutive successes to be considered healthy
`failureThreshold`	3	Consecutive failures before taking action

> **Time to failure:** `initialDelaySeconds + (periodSeconds × failureThreshold)` — e.g., 15 + (10 × 3) = **45 seconds** before restart.

HTTP Probe Deep Dive

```yaml
livenessProbe:
  httpGet:
    path: /healthz
    port: 8080
    scheme: HTTPS
    httpHeaders:
    - name: Authorization
      value: Bearer abc123
    - name: X-Custom-Header
      value: probe-check
  initialDelaySeconds: 10
  periodSeconds: 10
  timeoutSeconds: 3
```

- **Success:** HTTP status code ≥ 200 and < 400
- **port:** Container port number or named port reference
- **scheme:** HTTP (default) or HTTPS — does *not* verify certificates
- **httpHeaders:** Custom headers for auth or routing
- **path:** Dedicated health endpoint, not a business endpoint

🔌 TCP Probe Deep Dive

```yaml
spec:
  containers:
  - name: redis
    image: redis:7
    ports: [{ containerPort: 6379 }]
    readinessProbe:
      tcpSocket: { port: 6379 }
      initialDelaySeconds: 5
      periodSeconds: 10
    livenessProbe:
      tcpSocket: { port: 6379 }
      initialDelaySeconds: 15
      periodSeconds: 20
```

Best For: Databases, caches, message brokers — any service that doesn't speak HTTP.

Limitation: Only confirms the port is open — does not verify the application is functioning correctly.

Exec Probe Deep Dive

```yaml
spec:
  containers:
  - name: app
    image: myapp:1.0
    livenessProbe:
      exec:
        command:
        - /bin/sh
        - -c
        - |
          test -f /tmp/healthy &&
          find /tmp/healthy -mmin -1 | grep -q healthy
      initialDelaySeconds: 10
      periodSeconds: 15
      timeoutSeconds: 5
```

- **Exit code 0** = success, any other code = failure
- Runs *inside* the container — must have required binaries
- Useful for custom checks: file existence, process state, script validation
- Each exec probe creates a process — watch for **PID pressure** in high-frequency probes

🛰️ gRPC Probes

Native gRPC Health Checking (Kubernetes 1.24+)

Kubernetes can natively call the **gRPC Health Checking Protocol** without needing a sidecar or exec workaround.

- **service:** Optional — specifies which gRPC service to check (empty = overall status)
- App must implement the `grpc.health.v1.Health` service
- Returns `SERVING`, `NOT_SERVING`, or `UNKNOWN`

Configuration

```
livenessProbe:
  grpc:
    port: 50051
    service: "myservice"    # check specific service
  periodSeconds: 10
readinessProbe:
  grpc:
    port: 50051             # omit service = overall status
  periodSeconds: 5
```

🎯 Probe Best Practices

Patterns, Anti-Patterns, and Practical Guidance

⚠️ Common Anti-Patterns

🔴 Too Aggressive

- `periodSeconds: 1`
- `failureThreshold: 1`
- Restart loops on transient issues

🔴 Checking Dependencies

- Liveness checks DB connection
- DB down → all pods restart
- Cascading failure

🔴 Expensive Operations

- Heavy database queries
- Probe exceeds `timeoutSeconds`
- Causes false failures

🛠️ Probe Design Guidelines

💚 Liveness Probe Rules

- Check only *internal* application state
- Keep it lightweight (< 100ms)
- Use generous thresholds (`failureThreshold: 3+`)
- Never call external services
- Consider: "Would a restart fix this?"

🔵 Readiness Probe Rules

- Check dependencies if needed
- Verify cache is warmed
- Can be more thorough than liveness
- Consider: "Can I serve a good response?"
- Use `successThreshold: 2+` to avoid flapping

Golden Rule: Liveness answers "should I restart?" — Readiness answers "should I receive traffic?"

🌍 Example: Web Application

```
startupProbe:
  httpGet: { path: /healthz, port: 8080 }
  failureThreshold: 30
  periodSeconds: 5           # 150s max startup
livenessProbe:
  httpGet: { path: /healthz, port: 8080 }
  periodSeconds: 10
  failureThreshold: 3        # restart after 30s
readinessProbe:
  httpGet: { path: /ready, port: 8080 }
  periodSeconds: 5
  successThreshold: 2        # prevent flapping
```

> **Startup:** 150s max • **Liveness:** restart after 30s • **Readiness:** remove from LB within 15s

🚀 Real-World: Microservice Probes

gRPC Microservice

```
containers:
- name: order-svc
  image: order-service:1.5
  livenessProbe:
    grpc:
      port: 50051
    periodSeconds: 10
  readinessProbe:
    grpc:
      port: 50051
      service: "orders"
    periodSeconds: 5
```

- **Liveness:** Checks the gRPC server is responsive (overall status)
- **Readiness:** Checks the specific "orders" service is ready to handle requests

🚀 Real-World: Sidecar Probes

Database Sidecar (PgBouncer)

```
containers:
- name: pgbouncer
  image: pgbouncer:1.20
  livenessProbe:
    tcpSocket: { port: 6432 }
    periodSeconds: 15
  readinessProbe:
    exec:
      command: ["/bin/sh", "-c",
        "psql -h 127.0.0.1 -p 6432 -U probe -c 'SELECT 1'"]
    periodSeconds: 10
```

- **Liveness (TCP):** Is PgBouncer listening on port 6432?
- **Readiness (Exec):** Can it actually execute a query end-to-end?
- This layered approach catches both process-level and application-level failures

🔎 Debugging Probe Failures

Step 1: Check Pod Events

```
# See probe failure events
kubectl describe pod my-app | grep -A 5 "Events"
# Look for Warning events
kubectl get events --field-selector reason=Unhealthy \
  --sort-by='.lastTimestamp'
```

Step 2: Check Logs & Test Manually

```
# Current and previous container logs
kubectl logs my-app -c app --tail=50
kubectl logs my-app -c app --previous

# Exec into pod and test the health endpoint
kubectl exec my-app -- curl -s http://localhost:8080/healthz
kubectl exec my-app -- wget -qO- http://localhost:8080/ready
```

🛑 Graceful Shutdown & preStop Hooks

Pod Termination Sequence

1. Pod set to Terminating → **2.** preStop hook runs → **3.** SIGTERM sent → **4.** Grace period countdown → **5.** SIGKILL if still running

- **preStop + SIGTERM:** Happen concurrently with endpoint removal from Services
- **Race condition:** Add a `sleep 5` in preStop to let endpoints propagate
- **terminationGracePeriodSeconds:** Must exceed preStop + drain time

preStop Hook

```
terminationGracePeriodSeconds: 60
containers:
- name: app
  # ...
  lifecycle:
    preStop:
      exec:
        command: ["/bin/sh", "-c",
          "curl -s localhost:8080/shutdown; sleep 15"]
```

Pod Disruption Budgets (PDBs)

PDBs limit how many pods in a Deployment can be unavailable during *voluntary disruptions* (node drains, cluster upgrades, autoscaling).

Min Available

```yaml
apiVersion: policy/v1
kind: PodDisruptionBudget
metadata:
  name: web-pdb
spec:
  minAvailable: 2
  selector:
    matchLabels:
      app: web
```

Max Unavailable

```yaml
apiVersion: policy/v1
kind: PodDisruptionBudget
metadata:
  name: api-pdb
spec:
  maxUnavailable: "25%"
  selector:
    matchLabels:
      app: api
```

PDB Behavior & Interactions

- Works with `kubectl drain`, cluster autoscaler, and upgrade controllers
- Does **not** protect against involuntary disruptions (node crashes, OOM kills)
- Combine with readiness probes — unready pods count as unavailable

Probe interaction: A pod that fails its readiness probe counts as unavailable, which can block further voluntary disruptions. Always consider PDB + readiness probe behavior together.

🚀 Recommended Pod Spec Pattern

```
terminationGracePeriodSeconds: 45
containers:
- name: app
  image: myapp:2.0
  startupProbe:
    httpGet: { path: /healthz, port: 8080 }
    failureThreshold: 20
    periodSeconds: 5
  # ... livenessProbe, readinessProbe ...
  # ... resources, lifecycle, securityContext ...
```

📝 Quick Reference

Probe Type by Scenario

App hangs/deadlocks	Liveness
Cache warming / dependency down	Readiness
Slow JVM/ML model load	Startup
Memory leak detected	Liveness

Protocol by Workload

Web apps / REST APIs	HTTP
gRPC microservices	gRPC
Databases / Caches	TCP
Custom / legacy	Exec

🔍 Debugging Your Application

Common Failure States & Essential Troubleshooting Tools

🚨 Common Application Failure States

🔴 CrashLoopBackOff

- OOMKilled — memory limit exceeded
- Missing dependencies or binaries
- Configuration errors at startup

🔴 ImagePullBackOff

- Wrong image name or tag
- ECR auth failure or expired token
- Registry unreachable from EKS nodes

🔴 Pending Pods

- Insufficient CPU/memory on nodes
- Node affinity or taint mismatch
- PVC stuck in pending (EBS not bound)

CreateContainerConfigError: Missing ConfigMap or Secret referenced by the pod spec —

OOMKilled: Container exceeded its memory limit. Increase `resources.limits.memory`

🛠️ Essential Troubleshooting Tools

```
# Inspect pod status, events, and conditions
kubectl describe pod <pod-name>
kubectl describe node <node-name>

# View logs — current, previous crash, specific container
kubectl logs <pod> --tail=100
kubectl logs <pod> --previous
kubectl logs <pod> -c <container> --follow

# Interactive shell into a running container
kubectl exec -it <pod> -- /bin/sh

# Ephemeral debug container (no shell in distroless images)
kubectl debug -it <pod> --image=busybox --target=<container>

# Resource usage (requires metrics-server)
kubectl top pod     # CPU/memory per pod
kubectl top node    # CPU/memory per node

# Cluster events sorted by time
kubectl get events --sort-by=.lastTimestamp -n <namespace>
```

Workflow: Events ➔ Describe ➔ Logs ➔ Exec/Debug ➔ Top

🪦 Debugging Decision Tree

Symptom	First Command	What to Look For
Pod stuck **Pending**	`kubectl describe pod`	Events: FailedScheduling, resource requests, node selectors
Pod in **CrashLoopBackOff**	`kubectl logs --previous`	Stack traces, missing env vars, exit code (137=OOM, 1=app error)
ImagePullBackOff	`kubectl describe pod`	Events: image name typo, 401/403 from ECR, ImagePullSecrets
Pod **Running** but not working	`kubectl exec + logs`	App-level errors, connectivity to services, DNS resolution
High latency / restarts	`kubectl top pod`	CPU throttling (near limit), memory creeping toward limit

👨‍💻 Lab 10: Health Checks and Probes

Estimated Time: ~25-35 minutes

- Configure HTTP, TCP, and exec liveness probes
- Implement readiness probes and observe endpoint management
- Deploy startup probes for slow-starting applications
- Simulate application failures and observe probe-driven recovery
- Tune probe parameters and analyze detection timing
- Implement graceful shutdown with preStop hooks

> **Lab Guide:** Follow the step-by-step instructions in `labs/lab-10/README.md`

🎯 Key Takeaways

- **Liveness probes** detect stuck containers and trigger restarts — never check external dependencies
- **Readiness probes** control traffic routing — can and should check if the app can serve good responses
- **Startup probes** protect slow-starting apps from premature liveness kills
- **Probe protocols** include HTTP, TCP, gRPC, and exec for different workload types
- **Use all three together** for comprehensive health management
- **Tune parameters carefully** — too aggressive causes flapping, too relaxed delays recovery
- **Graceful shutdown** requires coordination between preStop hooks, SIGTERM handling, and PDBs
- **Recognize failure states:** CrashLoopBackOff, ImagePullBackOff, Pending, OOMKilled

Questions & Discussion

Health Probes & Application Debugging

Up Next: Module 11 — Deployment Strategies